Synopsys RTL to GDS Project Plan

Tool requirements, laboratory readiness and an execution plan for a digital ASIC project at Thapar Institute of Engineering and Technology.

Prepared for Kapil Tripathi | 9 October 2026 | Version 1.0

Begin by proving that one complete flow works on the college server. Confirm the tools, their licensed features, the technology package and the signoff decks together. Then use the same validated setup for the main project. This avoids discovering a missing dependency when the design reaches a later stage.

The recommended first project is a small, single clock, single voltage digital block. The result is a verified block layout and GDSII file with traceable reports. A full chip intended for fabrication needs additional chip integration and foundry requirements, described in Section 16.

Contents

1 Decide the scope and choose a route 2

2 Tools needed for the complete core flow 3

3 Conditional tools and licensed features 4

4 Technology files and library requirements 5

5 Questions to take to the Thapar lab 6

6 Software inventory to complete with the lab 7

7 Technology and server inventory 8

8 Prove the complete flow before starting 9

9 The complete implementation flow 10

10 Specification RTL and timing constraints 11

11 Synthesis equivalence and design for test 12

12 Floorplanning placement clocks and routing 13

13 Extraction signoff and final GDS 14

14 Suggested eight week execution plan 15

15 Repeatable runs and missing resource decisions 16

16 Final handoff and project completion 17

17 Official references and terminology 18

Take Sections 5 to 7 to the lab. Section 8 defines the evidence required before the main project starts. Sections 9 to 16 guide execution once the setup is ready.

1 Decide the scope and choose a route

Set the project boundary first

Ask the teacher to approve the design, its interfaces, the process technology and the required checks. Use a small synchronous FIFO implemented with registers, or an ALU with a register bank, for the first complete run. These are planning recommendations, not institutional requirements. Start without SRAM macros, multiple voltage domains or a large processor; each adds dependencies that must be validated separately.

Select the node that has a complete, working college flow. A mature training technology can be suitable for learning. A node name alone does not establish that its libraries, extraction files and verification decks are available.

Route A with separate synthesis and implementation

VCS → Design Compiler → IC Compiler II → extraction and signoff → final GDSII. Design Compiler produces the technology mapped netlist. IC Compiler II performs floorplanning, placement, clock tree synthesis and routing. Use the available Design Compiler edition and release that the laboratory reference flow supports. [1, 3]

This route is useful when the lab has an established Design Compiler and IC Compiler II kit. The separate stages also make intermediate artifacts easy to study.

Route B with Fusion Compiler

VCS → Fusion Compiler → extraction and signoff → final GDSII. Fusion Compiler integrates RTL synthesis and physical implementation in one environment. A separate Design Compiler or IC Compiler II run is not inherently required for this route. [2]

Confirm the signoff engines, integrations and feature licenses used by the lab kit. An integrated product name does not prove that every required operation is enabled for your account. Use the kit's documented signoff method; standalone PrimeTime, StarRC and IC Validator runs remain a clear way to collect final evidence.

Make one decision and record it

Decision	Record before starting
Project level	Block GDS demonstration, or full chip for a defined fabrication program
Implementation route	A with Design Compiler and IC Compiler II, or B with Fusion Compiler
Technology	PDK name, release, standard cell library and metal stack
Verification scope	Functional tests, equivalence, DRC, LVS, timing, power and any additional checks
DFT and advanced features	Included with tested resources, or excluded from this project by agreement

If only older IC Compiler is available, treat it as a separate legacy route with its own compatible library database and scripts. Do not reuse an IC Compiler II or Fusion Compiler recipe without validating it.

2 Tools needed for the complete core flow

The following is the laboratory request list for the recommended Synopsys flow. Every required role needs a named, tested solution. Products shown together in a role are alternatives only where stated.

Required role	Tool to ask for	Evidence that the role works
RTL and gate simulation	**VCS**	Compile and run a self checking testbench; open the resulting waveform. [4]
RTL synthesis	**Design Compiler**, or synthesis inside **Fusion Compiler**	Map a small design to the selected cell library and export its netlist and constraints. [1, 2]
Physical implementation	**IC Compiler II** after Design Compiler, or **Fusion Compiler**	Read the matched physical libraries; complete placement, CTS and routing; export a layout. [2, 3]
Formal equivalence	**Formality**	Prove the reference RTL and implemented functional netlist equivalent under documented assumptions. [6]
Interconnect extraction	**StarRC**, or the lab's validated signoff extraction integration	Extract the routed layout with the correct technology data and produce usable parasitics. [8]
Static timing signoff	**PrimeTime**, or a documented accepted integrated signoff run	Analyze the final netlist with parasitics and the agreed modes and corners. [7]
Physical verification	**IC Validator**	Execute the actual technology DRC and LVS decks on the exported layout. Check fill and antenna support as required. [9]

Formality is a requirement in the recommended verification plan. A limited teaching exercise can proceed without it only after the teacher changes the verification scope explicitly; simulation does not provide the same equivalence proof.

Two short lists to give the teacher

Route A: VCS, Design Compiler, IC Compiler II, Formality, PrimeTime, StarRC and IC Validator, with the matching technology package and a reference flow.

Route B: VCS, Fusion Compiler and Formality, with tested timing, extraction and physical verification capabilities. Ask for PrimeTime, StarRC and IC Validator access where those are used by the lab's signoff flow.

Helpful addition: Verdi for waveform and source level debug. VCS performs simulation; Verdi helps inspect and debug it. They are distinct capabilities. [4, 5]

Having a program on disk, or opening its GUI, is insufficient evidence. The proof is a completed run using the features and files required by this project.

3 Conditional tools and licensed features

Resolve these choices during the lab audit. A feature becomes required when the project uses it; removing it changes the agreed scope.

Capability	Synopsys tool or feature	When to include it
Waveform and source debug	**Verdi**	Recommended. Verify FSDB generation and loading when the flow uses FSDB. [5]
RTL quality and crossings	**VC SpyGlass Lint, CDC and RDC**, or the lab's supported SpyGlass flow	Recommended lint review. Dedicated crossing analysis is important when independent clocks or reset domains are present. Verify each module. [10]
Scan insertion	**TestMAX DFT** with the supported synthesis integration	Required if scan is part of the project. Confirm scan capable cells and test setup. [11]
Manufacturing test patterns	**TestMAX ATPG**	Required when generating fault model based test patterns and coverage reports. Scan insertion alone does not generate these patterns. [12]
Memory test	Supported **TestMAX memory test or BIST** flow	Needed when memory test is in scope. Confirm the exact module and the chosen memory's supported models.
Gate level power analysis	**PrimePower**, or a supported licensed power flow in the lab kit	Include for a measured power deliverable; validate activity input and reporting. Ask whether older scripts use PrimeTime PX. [13]
Power integrity and reliability	**RedHawk-SC** or an approved equivalent	Include for IR drop and electromigration signoff. This needs more than a total power estimate. [14]
Power intent verification	**VC LP** or the lab's supported low power verification flow	Needed for multiple power domains, isolation, retention or power switching. Confirm UPF support and licenses. [16]
Formal properties	**VC Formal**	Useful for assertions and protocol properties; it serves a different purpose from RTL to netlist equivalence. [15]

Dependencies inside the scripts

Check whether synthesis calls licensed DesignWare components, physical implementation invokes signoff integrations, or timing needs SI, advanced variation or distributed analysis features. Run those operations during the trial. Do not infer feature availability from the parent product's license.

Ask for prepared timing and physical libraries. **Library Compiler or library preparation tools** are needed only if you must build or convert those assets yourself. Library characterization tools such as PrimeLib are not normally needed when usable characterized cell libraries are supplied.

For a first block project, settle DFT, power reporting and IR drop scope with the teacher before implementation. Single clock designs still need careful handling of asynchronous inputs and reset release. A dedicated CDC license being absent does not make those risks disappear.

4 Technology files and library requirements

The process design kit describes a manufacturing technology. The standard cell library supplies actual gates and registers for that technology. Their releases, cell names and physical views must agree across the flow.

Asset to request	Why it is needed and what to verify
PDK identity and supported flow	Exact release, process option, metal stack, permitted project use and supported tool versions. Obtain the reference setup.
Timing and power libraries	Liberty `.lib` and the `.db` views required by the installed flow. Include all selected PVT corners and supported timing models. A Liberty file alone does not supply cell geometry.
Physical cell libraries	Prepared **NDM** reference libraries for the chosen IC Compiler II or Fusion Compiler flow, or the source physical and timing views plus a validated preparation script. [17]
Technology and abstract views	Tool compatible technology file or technology NDM; LEF technology and cell abstracts where the kit uses them. Check sites, routing layers, vias and units.
Cell layout views	Standard cell and macro GDSII or OASIS views for final streamout or merging. An abstract view is not a complete cell layout.
Routing RC technology	The exact TLU+, ITF or other RC setup required by implementation, including layer mapping and RC corners. File types depend on the kit.
Signoff extraction technology	StarRC compatible extraction data, such as the kit's `.nxtgrd` files, and its extraction run setup. Implementation RC estimates are not automatically the final extraction model.
Physical verification decks	Tool specific DRC, LVS, antenna, density and fill decks, plus required electrical rule checks and their configuration. An ICV license cannot execute an arbitrary Calibre deck.
LVS reference and cell models	Compatible netlist format, device and cell definitions, power pin conventions, permitted black boxes and include files.
Streamout mapping	Layer and datatype map, layout units, top cell convention, label layers and cell merging instructions.
Physical only and special cells	Tap, endcap, filler, tie, decap and antenna cells as applicable; clock buffers, scan cells and any power management cells required by the design.
IP and variation data	For macros: matched logical model, timing, abstract and layout views. For variation: approved derates or AOCV/POCV data when required.

Prefer the complete validated package over assembling unrelated downloads. Freeze the package release and cell library used for the trial. If SRAM is optional, begin with registers and add SRAM only after every required macro view is available. Full chip I/O and ESD libraries are additional requirements.

5 Questions to take to the Thapar lab

Request to the teacher or lab staff

I want to complete a digital RTL to GDSII project using the college's licensed Synopsys tools. Before choosing the design and schedule, I need to confirm one complete working flow, its technology files and access for my student account. Please help me fill the inventory in Sections 6 and 7 and run a small reference design through the required checks.

Ask these questions together

1. Which complete route is supported here: Design Compiler with IC Compiler II, Fusion Compiler, or an older validated flow? What exact releases and editions are installed?
2. Can my student account actually use the required synthesis, implementation, timing, extraction and DRC/LVS features? Are Formality and VCS available to it?
3. Which feature licenses are shared, how many simultaneous users can run, when do licenses expire, and how can I reserve time for long jobs?
4. Which PDK and standard cell library have already been used successfully? Can I access the matched NDM, timing libraries, layout views, RC data and verification decks?
5. Is there a known working end to end reference project, with scripts, reports and final GDS? Can I rerun it under my own account?
6. Are scan, ATPG, power analysis and IR drop checks expected in my project? Which licensed tools and data are available for each?
7. Which Linux server and OS release should I use? What are my CPU, memory, disk and runtime limits? Is there a scheduler or designated scratch area?
8. What setup script should I source, using which shell? How do remote login, GUI access, on-campus access and any approved off-campus access work?
9. Who handles tool problems, license failures and PDK problems? Please give one responsible contact for each and an agreed way to report failed jobs.
10. What is the expected final deliverable: a block GDS with reports, or a full chip for a particular fabrication program? Who reviews the final signoff scope?

Leave the meeting with concrete records

Obtain a chosen route, completed inventories, the reference project location, an account setup instruction and a date for the trial run. For every missing item, record the responsible person, proposed resolution and target date.

Ask for exact filenames, release identifiers and a demonstrated job rather than a general statement that Synopsys is available. Product brochures and course syllabi describe capabilities; the local inventory and trial establish what your project can use.

6 Software inventory to complete with the lab

Review date: _____ Lab or server: _____

Student account: _____ Teacher or lab contact: _____

Chosen route: A / B / validated legacy route **Reference project: _____**

Use **PASS** only after a job under the student account completes the required operation. Use **MISSING**, **UNTESTED** or **OUT OF SCOPE** for other cases. Record the exact licensed module; a product family name is insufficient.

Role and expected product	Exact tool release and launch path	Required feature tested and log location	Status
Simulation with VCS	_____	_____	_____
Debug with Verdi or chosen viewer	_____	_____	_____
Synthesis with DC or FC	_____	_____	_____
Implementation with ICC2 or FC	_____	_____	_____
Equivalence with Formality	_____	_____	_____
Timing with PrimeTime or validated integration	_____	_____	_____
Extraction with StarRC or validated integration	_____	_____	_____
DRC with IC Validator or approved equivalent	_____	_____	_____
LVS with IC Validator or approved equivalent	_____	_____	_____
Lint and any required CDC or RDC modules	_____	_____	_____
Scan insertion if included	_____	_____	_____
ATPG or memory test if included	_____	_____	_____
Power analysis if included	_____	_____	_____
IR drop and EM if included	_____	_____	_____

Shared license capacity and reservation method: _____

License expiry checked through the project period: _____

Feature or module restrictions: _____

Keep the full installation paths, feature names and logs in the project manifest. Complete this summary sheet from actual runs on the lab server.

7 Technology and server inventory

PDK and release: _____ Standard cell library: _____

Process option and metal stack: _____ Library release: _____

Resource	Location or identifier	Evidence or unresolved issue
Timing views and selected PVT corners	_____	_____
NDM or prepared physical libraries	_____	_____
Technology and RC layer maps	_____	_____
Cell and any macro layout views	_____	_____
Signoff extraction data	_____	_____
DRC and LVS decks and include files	_____	_____
Antenna, fill, density and any ERC decks	_____	_____
Streamout map and LVS reference setup	_____	_____
Required special cells and IP views	_____	_____
Working reference flow and script release	_____	_____

Access and compute

Supported Linux server and OS: _____

Login and remote GUI method: _____

Environment setup script and shell: _____

CPU and RAM allocation: _____ Disk and scratch quota: _____

Scheduler and maximum job time: _____ Backup location: _____

Off-campus access arrangement if needed: _____

Resolve all required gaps before the main project

Missing or failing item	Responsible person	Resolution and target date	Closed with evidence
_____	_____	_____	_____
_____	_____	_____	_____
_____	_____	_____	_____

Lab setup owner: _____ PDK contact: _____

Teacher scope agreement: _____ Trial rerun date: _____

Mark excluded features explicitly. A blank field means the requirement remains unresolved.

8 Prove the complete flow before starting

Run a small reference design on the actual lab server

Use a tiny register based block with a real clock, reset and data path. Run it under your student account, with the same versions, libraries, corners, decks and feature modules intended for the main project. A purely combinational example does not exercise CTS, and a GUI launch does not exercise the flow.

Trial step	Required evidence
Access and setup	A fresh login can load the approved environment, read the package and write to the project area. Record paths and tool banners.
RTL verification	Simulation ends with an explicit pass. Save a waveform. Exercise the chosen lint or crossing checks if included.
Synthesis and equivalence	Produce a mapped netlist and SDC; resolve library references; complete the agreed equivalence check.
Physical implementation	Complete floorplan, power connections, placement, CTS and routing using the selected libraries. Save checkpoints.
Extraction and timing	Read the final parasitics in timing analysis; check annotation, clocks, endpoints and the selected scenario matrix.
Physical verification	Run the real DRC and LVS decks on the exported layout, including required finishing checks. Explain all exclusions or waivers.
Extra project requirements	Exercise scan and ATPG, power, IR drop or low power checks when they are included. Verify their actual feature checkouts.
Final rerun	Export and reopen GDS; identify the top cell and referenced geometry. Repeat from a fresh environment using the saved scripts.

Conditions for starting the main project

- Every required role has a tested tool or a documented, tested substitute.
- All selected library views, extraction files and decks belong to a compatible package.
- The trial has no unexplained failed required checks. Logs distinguish a design problem from a missing dependency.
- The teacher has agreed the scope, target constraints, scenario matrix and any teaching-only limitations.
- License access, compute allocation and storage are expected to cover the planned period, with an escalation contact.
- A rerunnable flow, manifest and evidence directory have been saved.

Do not label an incomplete trial as a complete RTL to GDS flow. If a required resource is missing, resolve it or agree a smaller deliverable before the schedule starts. This audit prevents known dependency gaps; shared server outages and later license changes still need the recovery arrangements in Section 15.

9 The complete implementation flow

1	**Specification and verified RTL**	VCS and agreed RTL checks
↓		
2	**Mapped logic and optional scan**	Design Compiler or Fusion Compiler
↓		
3	**Equivalence and test validation**	Formality and any required TestMAX flow
↓		
4	**Floorplan and power network**	IC Compiler II or Fusion Compiler
↓		
5	**Placement and clock tree**	IC Compiler II or Fusion Compiler
↓		
6	**Routing and layout finishing**	IC Compiler II or Fusion Compiler
↓		
7	**Extraction and signoff closure**	StarRC, PrimeTime, IC Validator and scoped checks
↓		
8	**Final GDS and evidence archive**	Implementation export and final rechecks

Checks continue throughout implementation

Timing and physical checks begin before routing. Placement uses early estimates; the routed design supplies more accurate parasitics. When signoff finds a violation, make the appropriate engineering change order, or ECO, and repeat the affected extraction, timing, physical verification and equivalence checks.

Plan DFT before synthesis and floorplanning even when scan insertion is performed later in the logical flow. Preserve scan information through placement, chain reordering and final pattern generation. Record both functional and test modes when DFT is included.

Layout finishing is part of the flow

Insert the required physical cells, correct antenna issues, apply the approved fill and density strategy, and maintain power and well connectivity. If finishing changes geometry or electrical behavior, analyze the resulting final layout again. The final reports must describe the same revision as the delivered GDS.

The order of detailed commands depends on the validated PDK and tool kit. Use that kit for exact command syntax. This plan defines required stages, dependencies and evidence; it does not assume that scripts from another release will run unchanged.

10 Specification RTL and timing constraints

Define a design that can be checked

Write a short specification covering function, port widths, legal transactions, reset behavior and clock assumptions. For a synchronous FIFO, define depth, data width, full and empty behavior, simultaneous read and write, and the response to reset. Use a scoreboard or reference model so tests check results automatically.

Set the initial clock period, area objective and any power objective with the teacher and technology kit. Define the block's external timing assumptions. These values are project inputs; they cannot be chosen reliably from a tool name or node label.

Verify the RTL before expensive implementation

Run normal operation, reset, boundary cases, stalls, long sequences and relevant parameter settings. Review width and signedness errors, unintended latches, incomplete assignments and simulation warnings. Add assertions for the properties that matter. Where independent clocks, asynchronous inputs or reset domains exist, review synchronization and the appropriate CDC or RDC checks.

Save the RTL revision, source file list, test seeds, regression summary and representative waveforms. Coverage should reflect the verification plan rather than an arbitrary percentage. All intended tests must pass; warnings need an explanation or a fix.

Build the constraints and scenario matrix

Constraint or scenario item	Required decision
Clocks	Primary and generated clocks, period, waveform, uncertainty and relevant latency assumptions.
Interfaces	Input and output delays, reference clocks, driving cells or transitions and output loads.
Paths and resets	Valid exceptions with a reason; reset recovery/removal and any asynchronous behavior covered appropriately.
Electrical limits	Approved transition, capacitance and fanout constraints for the library and flow.
Modes	Functional mode, and any scan shift, scan capture or low power modes included in the project.
Corners	Required process, voltage, temperature, RC and variation settings for each mode. Use the kit's setup and hold combinations.

Exit evidence: approved specification, passing RTL regression, reviewed quality checks, a complete SDC and a teacher agreed scenario matrix. Review all unexplained unconstrained paths. Do not conceal missing constraints with broad false paths; do not assume one slow corner covers every setup check or one fast corner covers every hold check.

11 Synthesis equivalence and design for test

Synthesize using the frozen setup

Elaborate the design, link it to the selected libraries, apply constraints and map to legal technology cells. Confirm the top module, cell references, inferred memories, clock gating and any DesignWare use. Resolve unexpected black boxes and unintended latches. Record warnings and review timing, area and electrical constraints.

For Route A, export the mapped Verilog netlist and the constraints required by IC Compiler II. For Route B, save the Fusion Compiler design checkpoint and the netlist and reports needed by downstream checks. Capture the guidance files, such as SVF, required by the validated Formality flow.

Synthesis output: mapped netlist, propagated constraints, reports, tool and library identifiers, guidance files and a reproducible checkpoint. A pre-route timing result is an estimate; final timing is checked on the routed implementation.

Prove functional equivalence

Compare the RTL reference with the mapped functional design using Formality. Use consistent parameters, library models and documented reset or initialization assumptions. With scan present, apply the correct functional test-mode settings. Inspect unmatched, failing and unproven points rather than treating an interrupted or partial proof as a pass. [6]

Repeat equivalence after logic changing ECOs and other transformations that require it. Simulation and formal equivalence provide different evidence: the first exercises chosen behavior; the second checks the transformation under its stated assumptions.

Include the complete test branch when DFT is agreed

Plan scan ports, clocks, resets and functional/test modes early. Use the supported TestMAX DFT flow for test rule checking and scan insertion. Export the scan netlist, chain information and test protocol. Recheck functional equivalence with the proper mode constraints. TestMAX DFT supports scan structures; TestMAX ATPG provides pattern generation. [11, 12]

Maintain chain information through physical scan reordering. Run ATPG with the final chain connectivity and chosen fault models, then validate patterns using the supported simulation flow. Save fault coverage, untestable or aborted fault classifications, pattern counts and any accepted limitations. Add memory BIST only with a tested memory and test package.

Exit evidence: linked mapped design, reviewed synthesis reports, completed equivalence proof and, when included, validated scan and test artifacts. Define test coverage targets with the teacher; do not invent a universal acceptable threshold.

12 Floorplanning placement clocks and routing

Build the floorplan and power network

Load the mapped design or FC checkpoint with the matched physical libraries. Set the core and die dimensions appropriate to the project, placement rows, pin positions and routing resources. Place macros with legal orientations, halos and accessible pins when macros are included. Choose utilization from a feasible trial rather than maximizing density immediately.

Construct the power grid using the technology's approved rules. Connect power and ground to logic, macros and physical cells. Insert tap and endcap cells as required by the kit. Check connectivity and early power network quality; a drawn ring does not prove that every cell is powered.

Place and optimize

Place and legalize cells, then optimize timing, electrical constraints and congestion. Review the congestion map and timing endpoints. Check that required special cells and, if applicable, scan chain changes remain legal. Save a placement checkpoint and a report of any unresolved issues.

Create and analyze the clock tree

Run CTS using permitted clock cells and routing rules. Check skew, latency, clock transition and clock routing. Apply propagated-clock analysis at the appropriate stage of the kit. Review setup and hold with actual clock effects; also inspect recovery/removal and minimum pulse width where required.

Do not impose one arbitrary skew target on every technology. Use the agreed clock budget and the kit's rules. Resolve hold problems as well as setup problems after CTS.

Route and finish the layout

Perform global and detailed routing, then fix route connectivity, congestion and rule violations. Address antenna requirements using the kit's approved repairs. Finish with the required filler, decap, tie, density and metal fill operations in the sequence specified by the PDK flow. Recheck power connections and any scan reordering outputs.

Exit evidence: legal placed and routed design, reviewed clock report, no unexplained opens or shorts, preserved scan information if applicable, and checkpoints for floorplan, placement, CTS and routing. In-design route checks help implementation; the final physical verification decks and extracted timing still need to run.

13 Extraction signoff and final GDS

Analyze the final electrical design

Extract parasitics from the final geometry with the selected extraction technology. Verify units, layer mapping, coupling treatment, corner selection and annotation coverage. Read the matching netlist, libraries, SDC and parasitics into the agreed PrimeTime flow. Retain SPEF files such as `.spef` when used by the kit; generate SDF such as `.sdf` when timing annotated gate simulation is included. Review every required mode and corner. [7, 8]

Check setup, hold, relevant recovery/removal and pulse-width checks, electrical constraints and timing exceptions. Confirm no unexplained unconstrained endpoints or missing cells. Review signal integrity and variation using the methods required by the project. Timing signoff needs valid constraints and annotation as well as acceptable slack.

If power is included, analyze a defined workload and save the activity source, mapping coverage, analysis interval and leakage/dynamic results. State when results are vectorless estimates. For IR drop and electromigration, use the approved grid, current and reliability models with the selected power integrity flow. PrimePower and RedHawk-SC address these different roles. [13, 14]

Check the actual final layout

Check	What completion means
DRC	Required geometry, antenna and density checks completed with no unwaived violations; deck versions and any approved waivers recorded.
LVS	Extracted layout connectivity matches the correct reference, including power and ground conventions; no unexpected missing cells or black boxes.
ERC and reliability	Required electrical, well, supply and reliability checks completed for the agreed scope.
GDS integrity	Correct top cell, hierarchy, units, layer/datatype map and complete referenced cell geometry.

Export GDS using the approved mapping and merge instructions. Run the required final checks on that delivered layout, or follow a validated export verification procedure that proves it matches the checked database. Reopen it with a permitted layout viewer. A successful streamout command alone is insufficient.

Close the loop after a change

An ECO or new fill can invalidate earlier reports. Rerun affected extraction, timing, DRC/LVS, power, scan validation and equivalence checks as needed. Tie the final GDS, netlist and reports to one revision and record their hashes.

Exit evidence: passing required signoff reports and an archive that proves which exact layout they describe. Fabrication readiness also requires the program and foundry criteria in Section 16.

14 Suggested eight week execution plan

The schedule below is a planning estimate for a small block, assuming regular access and a complete validated setup. Start Week 1 only after the Section 8 trial passes. If time is limited, reduce design complexity while preserving the agreed flow and checks.

Period	Work and milestone	Evidence to retain
Before Week 1	Complete inventories, resolve required gaps, run and rerun the reference flow, freeze the manifest.	Trial GDS, passing required checks, scripts, versions, license and access plan.
Week 1	Approve block specification, interfaces, technology, clock target and verification scope. Prepare the project folders and test plan.	Specification, scenario matrix and teacher agreement.
Week 2	Write RTL and self checking tests; run lint and scoped crossing checks.	Passing regression, reviewed findings, RTL revision and waveforms.
Week 3	Create SDC, synthesize and prove equivalence. Perform the agreed DFT branch.	Mapped netlist, constraints, formal proof and any test artifacts.
Week 4	Import physical data; develop floorplan and power grid.	Readable checkpoint, power connectivity and feasibility checks.
Week 5	Place, legalize and optimize timing and congestion.	Placement checkpoint and reports showing feasible routing.
Week 6	Run CTS and routing; resolve clock, hold, antenna and route problems.	Routed checkpoint, clock reports and issue list.
Week 7	Finish layout; extract and close required timing and physical checks. Run scoped power and reliability checks.	Reports for the final geometry and any ECO evidence.
Week 8	Export and verify final GDS, rerun from the frozen setup and prepare the project report.	Complete handoff archive, demonstration and final review.

Use milestone completion to control progress

Review with the teacher at the setup trial, verified RTL, synthesis, routed design and final handoff milestones. Each review should inspect artifacts, not just screenshots. If a prerequisite fails, work on independent tasks while its owner resolves it; do not silently skip the required check.

Reserve access for implementation and signoff in advance, including rerun time. Keep margin for congestion changes and ECOs. Tool or PDK changes after the initial trial require another compatibility check before they become part of the main flow.

15 Repeatable runs and missing resource decisions

Freeze one documented setup

Maintain a manifest with tool releases and launch paths, the setup script, shell and server OS, licensed features, PDK and library releases, deck versions, RC corners, scenario matrix, source revision and file hashes. Keep sensitive license or account details in the approved local access record.

Use one environment setup script and one controlled configuration file for project paths. Keep tool scripts for simulation, synthesis, formal checking, implementation, extraction, timing and physical verification. These scripts must come from, or be adapted to, the validated kit; a folder name or generic sample command is not a runnable flow.

```
project/
  README.md             setup, validated run order and scope
  config/               manifest, paths and scenario matrix
  rtl/                  synthesizable sources and file lists
  tb/                   tests, assertions and reference models
  constraints/          SDC and UPF only when needed
  scripts/              validated stage scripts and launch wrappers
  runs/<revision>/      logs, reports and saved checkpoints
  results/<revision>/   final netlists, parasitics, GDS and summaries
```

Log every run. Check exit status, expected outputs and report status; some tools can return a file despite a failed required check. A preflight should detect changed tool releases, missing input files, inaccessible libraries and insufficient scratch space before long jobs start. Resume from the last compatible checkpoint and rerun affected checks after changes. Daily checks can be short once the full trial and manifest exist.

Decide substitutions before they disrupt the project

Problem	Decision to make during the audit
No Fusion Compiler	Use Route A if DC, ICC2 and the complete compatible signoff package pass the trial.
No ICC2 and no FC	Find a validated alternative implementation flow or resolve access. Synthesis alone cannot complete physical implementation.
ICV unavailable but Calibre available	Use the separately licensed, teacher approved Calibre DRC/LVS flow only with matching decks and a complete successful trial. [18]
Extraction or timing feature missing	Validate the kit's accepted alternative with the teacher; otherwise the required signoff role remains unresolved.
Debug, scan, power or memory feature missing	Validate a suitable substitute, or agree the corresponding scope change before starting. Avoid a memory macro with incomplete views.
Shared license or server unavailable	Use a reserved slot or a prevalidated backup host with the same accessible package. Follow the named escalation contact.

For layout inspection, a compatible viewer such as KLayout can help when available. A viewer does not replace physical verification. Once the route is frozen, additions and substitutions must be retested rather than introduced informally halfway through the project.

16 Final handoff and project completion

Deliver an archive that another person can reproduce

- Approved specification, verification plan and agreed project scope.
- Source RTL, testbench, file lists, passing test summary and representative waveforms.
- SDC, selected mode and corner matrix, and UPF when required.
- Frozen manifest, validated scripts, run instructions and relevant logs.
- Final mapped and implemented netlists, equivalence reports and checkpoints.
- Final parasitics, timing reports and explanation of any accepted exceptions.
- DRC, LVS and other required physical verification reports, with deck versions and approved waivers.
- DFT, ATPG, power and reliability artifacts when those checks are included.
- Final GDS, its top cell and layer map information, file hashes and a layout view.
- Short project report with the achieved frequency or timing target, area and supported power results. State assumptions and measured workloads.

Final completion checklist

[ ] The chosen route can be rerun under the student account from the recorded setup.

[ ] All required tests and proof obligations passed, with limitations explicitly documented.

[ ] Timing covers the agreed modes and corners with valid constraints and parasitic annotation.

[ ] The final exported layout passed the required physical checks.

[ ] GDS, netlist, parasitics and signoff reports refer to the same design revision.

[ ] Optional deliverables agreed with the teacher are included or formally removed from scope.

[ ] The archive is backed up and the teacher has reviewed the evidence.

If the goal is an actual chip

Before calling the result ready for fabrication, confirm the fabrication program, approved foundry kit and signoff requirements. Full chip work adds chip level integration, pad or bump planning, I/O and ESD cells, package interfaces, power delivery, test access, approved IP and any required seal ring or chip finishing structures.

Include the program's submission checklist, required data formats and authorized reviewer. A training PDK and a clean block GDS do not by themselves establish eligibility for manufacturing. Mask preparation, wafer fabrication, packaging and silicon test follow the design handoff and belong to the fabrication program.

The first block project should establish a complete, reproducible implementation flow. A later project can expand the design and chip scope after its extra resources pass the same readiness audit.

17 Official references and terminology

The references below identify tool roles and documented capabilities. Product and feature names were checked on **9 October 2026**. Use the installed release's documentation and the lab's validated kit for command syntax. The design choice, readiness criteria and schedule in this plan are project recommendations.

1. Synopsys Design Compiler NXT — RTL synthesis product family.
2. Synopsys Fusion Compiler — integrated RTL to GDSII implementation.
3. Synopsys IC Compiler II — placement, clocks, routing and implementation closure.
4. Synopsys VCS — simulation and functional verification.
5. Synopsys Verdi — waveform, source and verification debug.
6. Synopsys Formality — formal equivalence checking.
7. Synopsys PrimeTime — timing and related signoff analysis.
8. Synopsys StarRC — parasitic extraction.
9. Synopsys IC Validator — physical verification, including DRC, LVS and fill.
10. Synopsys VC SpyGlass — RTL lint, clock and reset crossing analysis.
11. Synopsys TestMAX DFT — design for test structures and scan implementation.
12. Synopsys TestMAX ATPG — test pattern generation and fault coverage.
13. Synopsys PrimePower — RTL through gate level power analysis.
14. Synopsys RedHawk-SC — voltage drop and electromigration analysis.
15. Synopsys VC Formal — formal property verification.
16. Synopsys low power design tools — low power simulation, static verification and power intent.
17. Synopsys Fusion Compiler design creation and synthesis training — library setup, NDM and scenario preparation.
18. Siemens Calibre physical verification and Calibre circuit verification — alternative physical verification roles, requiring the correct licenses and decks.

Terms used in the plan

DC Design Compiler; **ICC2** IC Compiler II; **FC** Fusion Compiler; **RTL** register transfer level; **GDSII** layout stream format; **PDK** process design kit; **NDM** Synopsys library and design database format; **SDC** Synopsys Design Constraints; **SPEF** Standard Parasitic Exchange Format; **SDF** Standard Delay Format; **PVT** process, voltage and temperature; **RC** resistance and capacitance; **STA** static timing analysis; **CTS** clock tree synthesis; **DRC** design rule checking; **LVS** layout versus schematic; **ERC** electrical rule checking; **DFT** design for test; **ATPG** automatic test pattern generation; **CDC** clock domain crossing; **RDC** reset domain crossing; **ECO** engineering change order; **UPF** Unified Power Format; **IR drop** supply voltage lost across resistance; **EM** electromigration.